

R Lab2. Anotaciones en R y búsquedas en Pubmed

Bioinformática. Ingeniería Biomédica

Anotaciones con Bioconductor

Instalar Bioconductor

Bioconductor contiene una colección de paquetes de R dedicados al análisis y comprensión de datos genómicos. Estas librerías contienen funcionalidades para una gran variedad de tareas relacionadas con la bioinformática.

Antes de instalar paquetes específicos de Bioconductor debemos instalar el paquete **BiocManager** desde el repositorio CRAN,

```
install.packages("BiocManager")
```

Para instalar paquetes específicos de Bioconductor utilizamos la función `install()` de **BiocManager**. Por ejemplo, para instalar el paquete **AnnotationDbi**:

```
BiocManager::install("AnnotationDbi",force=TRUE)
```

Conversión de identificadores de gen

Una tarea habitual que tendremos que hacer cuando analizamos datos bioinformáticos es convertir distintos tipos de identificadores de gen, que dependerán de la base de datos donde estén anotados y que no siempre tienen una correspondencia 1:1. Como ejemplo vamos a convertir identificadores de gen en Entrez (*Entrez gene ID*) en símbolos de gen (*gene symbol* según el *Human Genome Organisation (HUGO) Gene Nomenclature Committee*) y viceversa.

Necesitamos:

- Cargar el paquete **AnnotationDbi** de Bioconductor en nuestra sesión.
- Cargar el paquete de anotación referente al organismo con el que vamos a trabajar (*OrgDb*). En este caso trabajamos con ser humano, por tanto **org.Hs.eg.db**, que cargara en nuestra sesión de R la anotación en Entrez de ser humano.
- Un listado de genes, sus identificadores *Entrez IDs*, que queremos convertir a *gene symbol*. Crea un objeto de tipo **vector** con estos identificadores: 1, 10, 100, 1000 y 37690.

En un paquete de tipo organismo (*OrgDb*) hay varios tipos de anotaciones disponibles que podéis consultar con la instrucción,

```
ls("package:org.Hs.eg.db")
```

```
## [1] "org.Hs.eg"                "org.Hs.eg.db"
## [3] "org.Hs.eg_dbconn"         "org.Hs.eg_dbfile"
## [5] "org.Hs.eg_dbInfo"         "org.Hs.eg_dbschema"
## [7] "org.Hs.egACCNUM"          "org.Hs.egACCNUM2EG"
## [9] "org.Hs.egALIAS2EG"        "org.Hs.egCHR"
## [11] "org.Hs.egCHRLNGTHS"       "org.Hs.egCHRLOC"
## [13] "org.Hs.egCHRLOCEND"       "org.Hs.egENSEMBL"
## [15] "org.Hs.egENSEMBL2EG"      "org.Hs.egENSEMBLPROT"
```

```
## [17] "org.Hs.egENSEMBLPROT2EG" "org.Hs.egENSEMBLTRANS"
## [19] "org.Hs.egENSEMBLTRANS2EG" "org.Hs.egENZYME"
## [21] "org.Hs.egENZYME2EG" "org.Hs.egGENENAME"
## [23] "org.Hs.egGENETYPE" "org.Hs.egGO"
## [25] "org.Hs.egGO2ALLEGS" "org.Hs.egGO2EG"
## [27] "org.Hs.egMAP" "org.Hs.egMAP2EG"
## [29] "org.Hs.egMAPCOUNTS" "org.Hs.egOMIM"
## [31] "org.Hs.egOMIM2EG" "org.Hs.egORGANISM"
## [33] "org.Hs.egPATH" "org.Hs.egPATH2EG"
## [35] "org.Hs.egPFAM" "org.Hs.egPMID"
## [37] "org.Hs.egPMID2EG" "org.Hs.egPROSITE"
## [39] "org.Hs.egREFSEQ" "org.Hs.egREFSEQ2EG"
## [41] "org.Hs.egSYMBOL" "org.Hs.egSYMBOL2EG"
## [43] "org.Hs.egUCSCCKG" "org.Hs.egUNIPROT"
```

En este paquete, `org.Hs.eg.db`, la anotación central es la de Entrez (`eg`). Así, por ejemplo, podríamos convertir *Entrez IDs* a *RefSeq* (`org.Hs.egREFSEQ`) o *Ensembl IDs* (`org.Hs.egENSEMBL`) y viceversa (`org.Hs.egREFSEQ2EG` y `org.Hs.egENSEMBL2EG`, respectivamente). En este último caso, se añade el sufijo `...2EG`.

Queremos obtener la correspondencia *Entrez IDs* a *gene symbol*, ¿cuál sería la anotación a utilizar? ¿Y para transformar *gene symbol* en *Entrez IDs*?

La función `AnnotationDbi::mget()` permite hacer las conversiones entre identificadores y requiere de dos argumentos: el vector con los identificadores a traducir y una anotación. Además, podemos utilizar el argumento `ifnotfound` para asignar NA a aquellos identificadores que no se encuentran. Haz la conversión de *Entrez ID* a *gene symbols* utilizando `AnnotationDbi::mget()`.

La función `mget()` devuelve una lista cuyos elementos se nombran con el correspondiente identificador de partida, *Entrez ID* en este caso (`myEIDs`). Convierte la lista en un **vector**.

Ahora vamos a volver a los *Entrez IDs*. El primer paso será seleccionar los identificadores informativos, es decir los distintos de NA, y después volver a utilizar la función `mget()`, con la anotación que corresponda.

Anotaciones en GO

Gene Ontology (GO) proporciona información sobre las características de genes y productos génicos a través de su anotación. Los términos GO recogen lo que se conoce de los genes en términos de su función, fisiología y localización celular, proporcionando una buena descripción a nivel biológico. Veamos como mapear un conjunto de genes en los términos GO en los que están anotados. Necesitaremos:

- tener instalado y cargado el paquete `GO.db`.
- cargar el paquete de anotación del organismo que corresponda. Para ser humano `org.Hs.eg.db`.

Queremos obtener la correspondencia Entrez IDs a la(s) categoría(s) GO en las que están anotados, ¿cuál sería la anotación a utilizar? ¿Y para transformar GO en Entrez IDs?

De gen a GO

Partimos de una lista de identificadores *Entrez IDs* de genes que queremos anotar en GO, por ejemplo el gen 1, 10 y 100. **Nota:** Es conveniente que el punto de partida sea ese indentificador Entrez, si no es así, siempre podéis convertir el listado como hemos hecho anteriormente. Encontrar la anotación GO del segundo gen.

Podemos utilizar la función `mget()` para obtener la anotación de todos los genes,

```
myGO_All <- mget(myEIDs, org.Hs.egGO)
```

Contruye un vector con los identificadores de todas las categorías GO en las que están anotados esos genes. Nota: podeis utilizar la función `names()` para recuperar los nombres de elementos de una lista que le pasais como argumento y `lapply()` para repetir `names()` en cada elemento de la lista.

De GO a gen

A partir de un identificador GO también podemos obtener los genes anotados en esa categoría. Para obtener los *Entrez ID* de los genes anotados en las categorías “GO:0008150” y “GO:0001909”, utiliza `mget()` con la anotación que corresponda.

PubMed en R

En esta segunda parte del laboratorio, vamos a trabajar con información sobre las publicaciones en PubMed utilizando R.

PubMed es una de las bases de datos del *National Center for Biotechnology Information* (NCBI), de acceso libre y especializada en ciencias de la salud, con más de 19 millones de referencias bibliográficas a la que se puede acceder a través de Entrez (*Global Query Cross-Database Search System*).

PubMed contiene las referencias de MEDLINE. MEDLINE es una base de datos de citas y abstracts de acceso online y gratuito, de carácter médico.

Utilizamos el paquete `RISmed` disponible en el CRAN de R para llevar a cabo las consultas a las bases de datos. Además utilizaremos el paquete `rentrez` para obtener información acerca del número de citas de estos artículos.

Vamos a ver un ejemplo que se centra en la búsqueda artículos relacionados con **BRCA1 para cáncer de mama en humanos durante el año 2024**.

Instalar y cargar las librerías

Instala y carga las librerías que vamos a utilizar, todas ellas disponibles en CRAN: `RISmed`, `dplyr`, `rentrez`, `ggplot2`, `PubMedWordcloud` y `wordcloud2`.

Búsqueda en PubMed

El primer paso será establecer los términos de búsqueda. Crea un vector de longitud 1 y de tipo `character` en el que almacenes el texto de la consulta. Queremos buscar publicaciones que contengan en su título ‘BRCA1’ y ‘Breast Cancer’, publicadas en ‘2024’. El formato del *string* es el mismo que utilizaríamos para hacer la consulta directamente en PubMed.

Nota: En el enlace https://www.nlm.nih.gov/bsd/licensee/data_elements_doc.html?utm_source=chatgpt.com tenéis disponible la descripción de los principales campos que podéis utilizar para construir el *string* de búsqueda en PubMed.

Para extraer la información utilizamos las funciones `RISmed::EUtilsSummary()` y `RISmed::EUtilsGet()`. Con la primera obtenemos información resumen sobre la búsqueda que se va a llevar a cabo, y con `RISmed::EUtilsGet()` obtenemos los resultados.

- La función `RISmed::EUtilsSummary()` tiene como primer argumento el *string* de búsqueda creado en el paso anterior. Otros argumentos a utilizar son: `type="esearch"` y `db="pubmed"`, que limitan la búsqueda a Pubmed.
- La función `RISmed::EUtilsGet()` requiere como primer argumento el objeto creado con la función `RISmed::EUtilsSummary()`. Otros argumentos a utilizar son: `type = "efetch"` y `db = "pubmed"`.

Crea un objeto con los resultados de la búsqueda. ¿De qué clase es el objeto que habéis creado? ¿Cuántas publicaciones cumplen los criterios de búsqueda?

Conjunto de datos

Vamos a hacer un análisis estadístico con los datos que hemos obtenido, pero previamente debemos construir un `data.frame` con la información que vamos a utilizar de cada publicación. Podemos recuperar la información de cada campo utilizando funciones específicas, concretamente en este caso nos va a interesar los siguientes:

1. `PMID()` que devuelve el identificador del artículo en PubMed.
2. Fecha de la publicación con dos campos: mes y día. Utilizamos las funciones `MonthEntrez()` y `DayEntrez()`, respectivamente.
3. Título, con la función `ArticleTitle()`.
4. El identificador DOI (Digital Object Identifier) del artículo (`DOI()`).
5. País (`Country()`)
6. El tipo de publicación (`PublicationType()`).

Podéis consultar todos los campos que podéis obtener en la ayuda de la clase `Medline`.

```
help("Medline")
```

¿Se ha podido obtener toda la información?

Añadiendo información del número de citas con `rentrez` Vamos a añadir la información del número de citas de cada uno de los artículos que hemos obtenido en nuestra consulta utilizando el paquete `rentrez`, conocido el identificador del artículo en PubMed (`pmid`).

Para obtener el número de citas de un artículo con identificador '36553480' ejecutaríamos el siguiente código,

```
pmid <- "36553480"
pmid_summary <- entrez_summary(db="pubmed", id=pmid)
as.numeric(extract_from_esummary(pmid_summary, "pmcrefcount"))
```

```
## [1] 41
```

Generaliza este código para obtener el número de citas de todos los artículos que cumplen nuestros criterios de búsqueda. ¿Qué significa en este caso que este campo esté vacío?

Añade el número de citas al `data.frame` que contiene el resto de la información.

El paquete `dplyr` no proporciona ninguna nueva funcionalidad a R per se, en el sentido que todo aquello que podemos hacer con él lo podríamos hacer con la sintaxis básica de R. Sin embargo, es especialmente útil porque proporciona una “gramática” que facilita la manipulación y operaciones con data frames. Proporciona una serie de funciones, conocidas como “verbos”, que permiten realizar operaciones comunes de manera intuitiva y legible, por ejemplo `filter()`, `select()` o `summarize()`. Estas funciones pueden encadenarse utilizando el operador pipe (`%>%`), lo que permite construir secuencias de operaciones de manera clara y concisa.

Por ejemplo,

```
pubmed_data <- pubmed_data %>% mutate(title=substr(title,1,20)) %>% # reducimos a 20 caracteres
  dplyr::distinct(pmid,.keep_all = TRUE) # seleccionamos los pmid distintos
pubmed_data[1,]
```

```
##      pmid monthPub dayPub      title doi language country
## 1 39730675      12     27 PARP inhibition radi <NA>      eng      <NA>
##      pubType num.citas
## 1 Journal Article      57
```

La función `dplyr::mutate()` sirve para añadir nuevas variables y conservar algunas ya existentes en el data frame `pubmed_data`. La función `base::substr()` extrae los caracteres desde la posición `start` hasta `stop`. Con `dplyr::distinct()` seleccionamos las filas distintas de `pubmed_data` en la variable `pmid`. Usamos el argumento `.keep_all=TRUE` para mantene en el `data.frame` resultado todas las variables de `pubmed_data`.

Vamos a utilizar el paquete DT disponible en CRAN para mostrar la información de una forma más amigable, en forma de tabla interactiva:

```
# para presentarlo en forma de tabla necesitamos el paquete DT
if(!("DT" %in% installed.packages()))
  install.packages("DT")
library("DT")

pubmed_data %>% DT::datatable()
```

Utilizamos la función `DT::datatable()` para crear una tabla en html con el `data.frame` resultado.

Show 10 entries		Search:							
	pmid	monthPub	dayPub	title	doi	language	country	pubType	num.citas
1	38098088	12	15	Prevalence of BRCA1,		eng		Journal Article	74
2	38093030	12	13	Cost-Effectiveness A		eng		Journal Article	78
3	38057686	12	6	Presence of crown-li		eng		Journal Article	0
4	38053165	12	5	Prenatal BRCA1 epimu		eng		Journal Article	62
5	38030749	11	29	Risk-reducing mastec		eng		Journal Article	17

Resumen de los datos

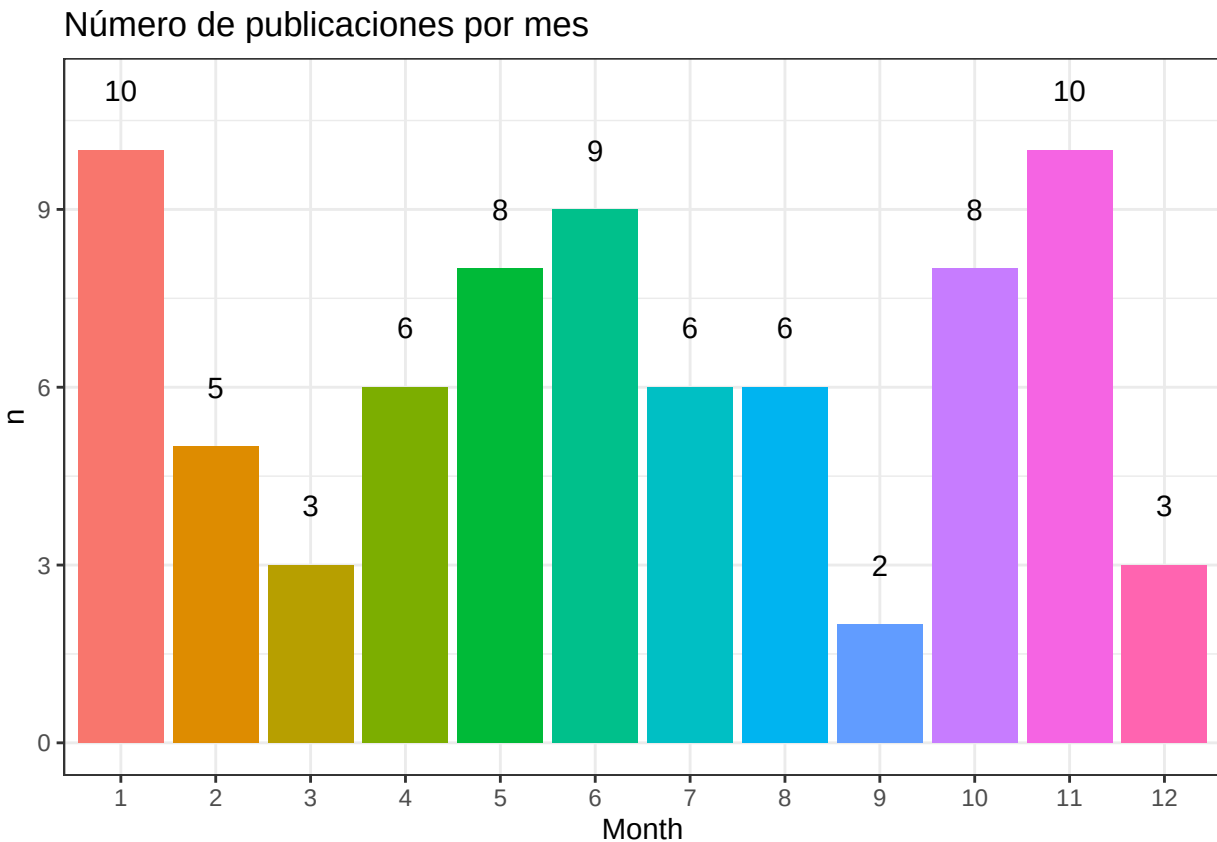
Utilizamos el paquete `ggplot2` para construir gráficos. Este paquete permite elaborar un gráfico a partir de un proceso de acumulación de capas o “layers”. Tiene un cierto nivel de complejidad pero se obtienen resultados muy profesionales. Lo básico de este paquete lo podéis encontrar en <http://www.cookbook-r.com/Graphs/>.

Las capas se van combinando con el signo `+`. Las más relevantes:

- Un data frame y las variables a representar
- El tipo de objeto(s) a representar o `geom` (barra, línea, punto, etc.)
- Es posible personalizar cada detalle de un gráfico, como los colores, tipos de líneas, fuentes o alineación, entre otros, usando los componentes de la función `theme`.
- Otras funciones que permiten añadir títulos, subtítulos, líneas, flechas o textos.

Con el siguiente código generamos un **gráfico de barras** en el que representamos el **número de publicaciones por mes**,

```
pubmed_data %>% group_by(Month=factor(monthPub)) %>%  
  summarise(n=n()) %>%  
  ggplot(aes(x=Month,y=n,fill=Month)) +  
  geom_bar(stat = "identity") +  
  geom_text(aes(label=n),nudge_y = 1) +  
  theme_bw() + theme(legend.position = "none") +  
  ggtitle("Número de publicaciones por mes")
```



En las dos primeras líneas se utiliza funciones del paquete `dplyr` para crear un nuevo data frame a partir de `pubmed_data` que contiene el número de las publicaciones distintas cada mes.

```
pubmed_data %>% group_by(Month=factor(monthPub)) %>%  
  summarise(n=n())
```

```
## # A tibble: 12 x 2  
##   Month     n
```

```
##      <fct> <int>
##  1  1      10
##  2  2       5
##  3  3       3
##  4  4       6
##  5  5       8
##  6  6       9
##  7  7       6
##  8  8       6
##  9  9       2
## 10 10       8
## 11 11      10
## 12 12       3
```

Las siguientes líneas son funciones del paquete **ggplot2** que generan el gráfico a partir de ese nuevo data frame.

- Con **ggplot()** indicamos las variables que vamos a utilizar: en el eje de las X el mes, en el eje Y el número de publicaciones y el color será distinto para cada mes.
- **geom_bar()** indica que queremos un gráfico de barras.
- Con **geom_text()** añadimos el texto encima de la barra.
- **theme_bw()** para indicar un tipo de aspecto del gráfico preconfigurado distinto del clásico, optimizado para proyector.
- Con **theme()** eliminamos la leyenda.
- Y por último, con **ggtitle()** añadimos un título.

Utiliza **coord_polar(theta = "y")** para representar en un **gráfico de sectores** en el que representamos el **número de citaciones por tipo de publicación**.

Número de citaciones por tipo de publicación

